EASY TASKS (≈30%)

1. Core Concepts – Pod

Task:

Create a pod named **busybox-pod** using the **busybox:latest** image that executes the command:

```
sleep 3600
```

2. Core Concepts – Deployment

Task:

Create a deployment named **web-deploy** using the **nginx:1.27** image with **3 replicas**.

3. Core Concepts – ReplicaSet

Task:

Create a ReplicaSet named **frontend-rs** with **4 replicas** using the **nginx:latest** image.

Pods must have the label:

```
app: frontend
```

4. Core Concepts – ClusterIP Service

Task:

Expose the deployment **web-deploy** internally using a **ClusterIP** service named **web-svc** on port **80**.

5. Core Concepts – NodePort Service

Task:

Expose the deployment **web-deploy** using a **NodePort** service named **web-nodeport**.

The service should expose port **80**.

6. Core Concepts – Labels & Selectors

Task:

Create a pod named **redis-pod** running **redis:latest**.

Assign the label:

```
tier=cache
```

7. Scheduling – Taints & Tolerations

Task:

A node named **worker1** has the taint:

```
dedicated=db:NoSchedule
```

Create a pod named **db-pod** that can be scheduled onto this node.

8. Scheduling – NodeSelector

Task:

Node **worker2** has the label:

```
disk=ssd
```

Create a pod named **storage-pod** that schedules only on nodes with this label.

9. Scheduling – Node Affinity

Task:

Create a pod named **analytics-pod** that schedules only onto nodes labeled:

```
cpu=high
```

Use **requiredDuringSchedulingIgnoredDuringExecution** node affinity.

10. Workloads – Scaling

Task:

Scale the deployment **web-deploy** from **3 replicas** to **6 replicas**.

11. Workloads – Logs

Task:

Display the logs of the pod **busybox-pod**.

12. Workloads – Exec

Task:

Execute an interactive shell inside the pod **busybox-pod**.

13. Workloads – Liveness Probe

Task:

Create a pod named **health-pod** running **nginx**.

Configure a liveness probe that performs an HTTP GET request on:

/

using port **80**.

14. Workloads – Readiness Probe

Task:

Create a pod named **ready-pod** running **nginx**.

Configure a readiness probe using HTTP GET on port **80**.

15. Config – ConfigMap

Task:

Create a ConfigMap named **app-config** containing:

`MODE=production`

16. Config – Secret

Task:

Create a secret named **db-secret** containing:

`password=MyPass123`

17. Config – Mount ConfigMap as Environment Variable

Task:

Create a pod named **config-pod** using **busybox**.

Inject the value of **MODE** from the ConfigMap **app-config** as an environment variable.

18. Config – Mount Secret as Volume

Task:

Create a pod named **secret-pod**.

Mount the secret **db-secret** at:

`/etc/secret`

19. Cluster – View Nodes

Task:

Display all nodes in the cluster with their labels.

20. Cluster – View Namespaces

Task:

Display all namespaces in the cluster.

21. Cluster – Switch Context

Task:

Switch kubectl to use the context:

`cluster2`

MID TASKS (≈40%)

22. Storage – PersistentVolume

Task:

Create a PersistentVolume named **pv-data** with:

- 2Gi storage
- ReadWriteOnce
- hostPath `/data/pv`
- storageClassName `manual`

23. Storage – PersistentVolumeClaim

Task:

Create a PersistentVolumeClaim named **data-pvc** requesting:

- 1Gi storage
- ReadWriteOnce
- storageClassName `manual`

24. Storage – Mount PVC

Task:

Create a pod named **app-pod** using **nginx**.

Mount the PVC **data-pvc** at:

`/usr/share/nginx/html`

25. Networking – Deny All NetworkPolicy

Task:

Create a NetworkPolicy named **deny-all** in namespace **default** that blocks all ingress and egress traffic for every pod.

26. Networking – Allow by Label

Task:

Create a NetworkPolicy allowing ingress traffic only from pods labeled:

`role=frontend`

to pods labeled:

`app=backend`

27. Networking – Allow by Port

Task:

Create a NetworkPolicy allowing ingress traffic on TCP port **8080** only.

28. Networking – Service & Endpoints

Task:

Create a ClusterIP service named **backend-svc** exposing pods labeled:

```
app=backend
```

on port **80**.

Verify that endpoints are created.

29. Security – ServiceAccount

Task:

Create a ServiceAccount named **developer-sa**.

30. Security – Role

Task:

Create a Role named **pod-reader** in the **default** namespace.

The role should allow:

- get
- list

on pods.

31. Security – RoleBinding

Task:

Bind the Role **pod-reader** to the ServiceAccount **developer-sa**.

32. Security – Verify Permissions

Task:

Verify whether the ServiceAccount **developer-sa** can list pods in the default namespace.

33. Maintenance – ResourceQuota

Task:

Create a ResourceQuota named **rq-dev** in namespace **dev** limiting the total requested CPU to **2** and memory requests to **2Gi**.

34. Maintenance – LimitRange

Task:

Create a LimitRange named **default-limits** in namespace **dev**.

Set default CPU request to **200m** and default memory request to **256Mi**.

35. Workload Management – Rolling Update

Task:

Update deployment **web-deploy** from **nginx:1.27** to **nginx:1.28**.

Wait until the rollout completes successfully.

36. Workload Management – Rollback

Task:

Undo the most recent rollout of deployment **web-deploy**.

37. Workload Management – Rollout History

Task:

Display the rollout history for deployment **web-deploy**.

38. Workload Management – Rolling Update Strategy

Task:

Modify deployment **web-deploy** so that its rolling update strategy uses:

```
maxUnavailable: 1
maxSurge: 2
```

39. Context Switching

Task:

Before performing any changes, switch kubectl to the context:

```
production-cluster
```

Then create a pod named **test-pod** using the **nginx** image.